Shlok Hemnani
D15C/62

ML&DL Experiment No.3

AIM:

Apply Decision Tree and Random Forest for classification tasks.

---

1. Dataset Source

- **Dataset Name:** Drug200 Dataset
- **Source:** Kaggle
- **Type:** Multiclass Classification Dataset

---

2. Dataset Description

The dataset contains information about patients and the type of drug prescribed to them based on medical conditions.

- **Size:** 200 samples × 6 attributes
- **Target Variable:** Drug (DrugA, DrugB, DrugC, DrugX, DrugY)
- **Type:** Multiclass classification problem

Features (5 predictors):

1. **Age:** Age of the patient
2. **Sex:** Male/Female

3. **BP:** Blood Pressure level (HIGH, NORMAL, LOW)
4. **Cholesterol:** Cholesterol level (HIGH, NORMAL)
5. **Na_to_K:** Sodium to Potassium ratio in blood

---

3. Mathematical Formulation of the Algorithm

A. Decision Tree (CART Algorithm)

A Decision Tree splits the dataset into subsets based on the feature that produces the purest child nodes.

Gini Impurity:

Gini=1−∑pi2Gini = 1 - \sum p_i^2Gini=1−∑pi2

Where:
pip_ipi = Probability of class i

Lower Gini value indicates better split.

Entropy:

Entropy=−∑pilog2(pi)Entropy = - \sum p_i \log_2(p_i)Entropy=−∑pilog2(pi)

Information Gain:

IG=Entropy(parent)−Weighted Entropy(children)IG = Entropy(parent) - Weighted\ Entropy(children)IG=Entropy(parent)−Weighted Entropy(children)

The feature with highest Information Gain is selected for splitting.

B. Random Forest (Ensemble Learning)

Random Forest builds multiple decision trees and combines their outputs.

Key Concepts:

- **Bagging:** Random sampling with replacement.

- **Feature Randomness:** Random subset of features at each split.
- **Voting:** Final prediction = Majority vote of all trees.

This reduces overfitting and increases model stability.

---

4. Algorithm Limitations

Decision Tree:

- Prone to overfitting
- Sensitive to small data changes
- High variance

Random Forest:

- Computationally expensive
- Less interpretable (Black Box Model)
- Requires more memory

---

5. Methodology / Workflow

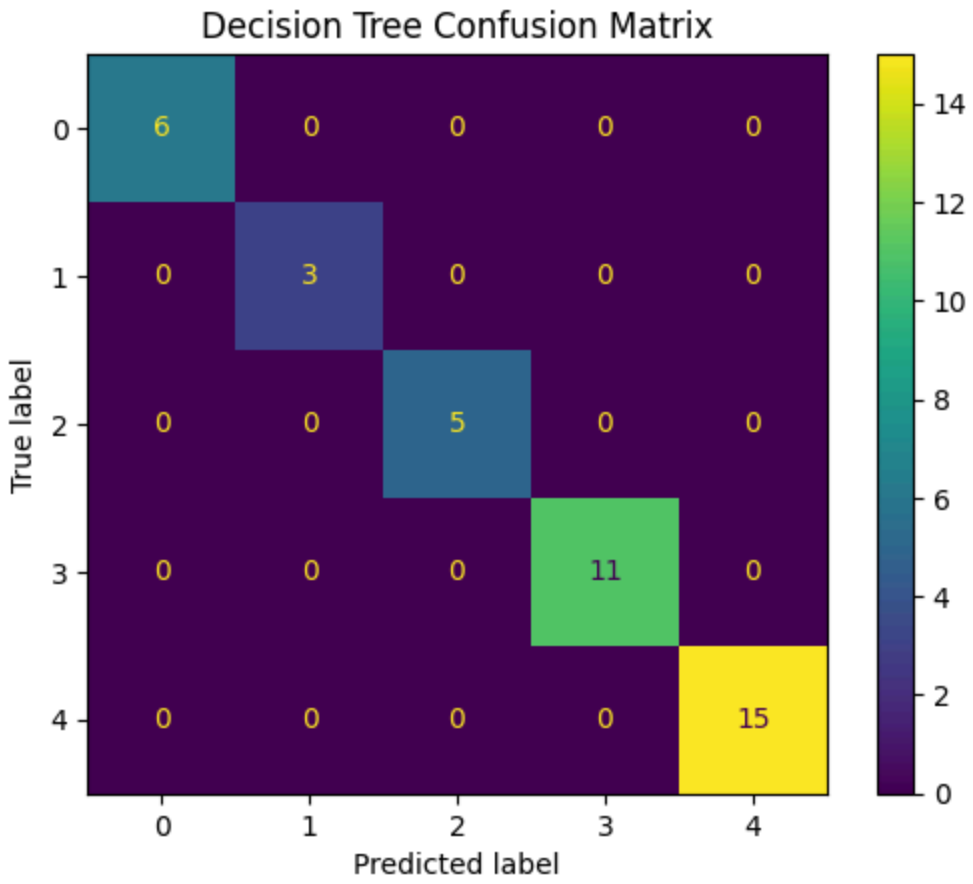
1. Import libraries
2. Load dataset (drug200.csv)
3. Encode categorical variables
4. Split into training and testing sets
5. Train Decision Tree model

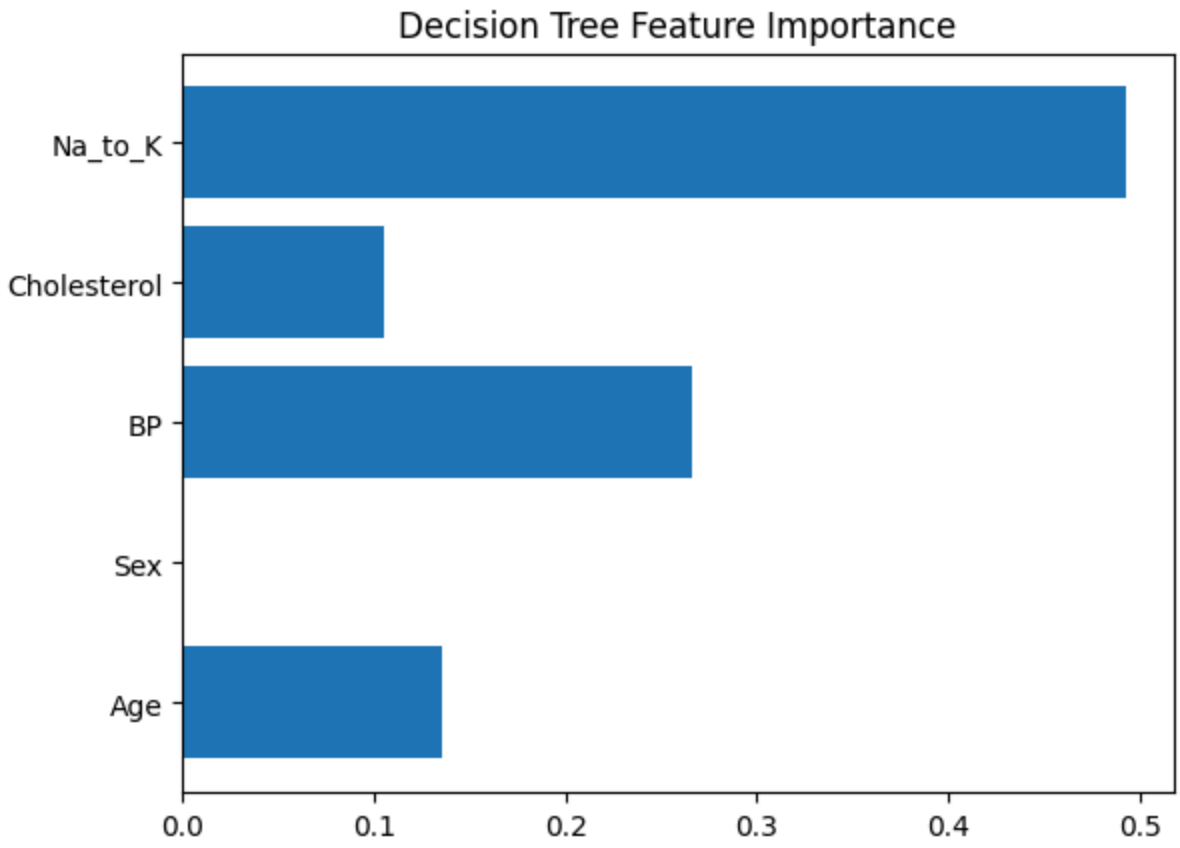
6. Train Random Forest model
7. Evaluate performance
8. Perform hyperparameter tuning
9. Compare results

6. Code and Output

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection
import train_test_split

from sklearn.preprocessing
import LabelEncoder,
StandardScaler

from sklearn.tree import
DecisionTreeClassifier

from sklearn.ensemble import
RandomForestClassifier

from sklearn.metrics import
accuracy_score,
confusion_matrix,
ConfusionMatrixDisplay

# ========================

# PART 1 — DECISION TREE (DRUG
DATASET)

# ========================

drug_df =
pd.read_csv('drug200.csv')

# Encode categorical columns

le = LabelEncoder()

for col in drug_df.columns:

if drug_df[col].dtype ==
'object':

drug_df[col] =
le.fit_transform(drug_df[col])

X_drug = drug_df.drop('Drug',
axis=1)

y_drug = drug_df['Drug']

X_train_d, X_test_d, y_train_d,
y_test_d = train_test_split(

X_drug, y_drug, test_size=0.2,
random_state=42

)

dt_model =
DecisionTreeClassifier()

dt_model.fit(X_train_d,
y_train_d)

y_pred_d =
dt_model.predict(X_test_d)

print("Decision Tree
Accuracy:",
```

```
accuracy_score(y_test_d,
y_pred_d))

# Confusion Matrix

cm_d =
confusion_matrix(y_test_d,
y_pred_d)

ConfusionMatrixDisplay(cm_d).pl
ot()

plt.title("Decision Tree
Confusion Matrix")

plt.show()

# Feature Importance

plt.figure()

plt.barh(X_drug.columns,
dt_model.feature_importances_)

plt.title("Decision Tree
Feature Importance")

plt.show()
```

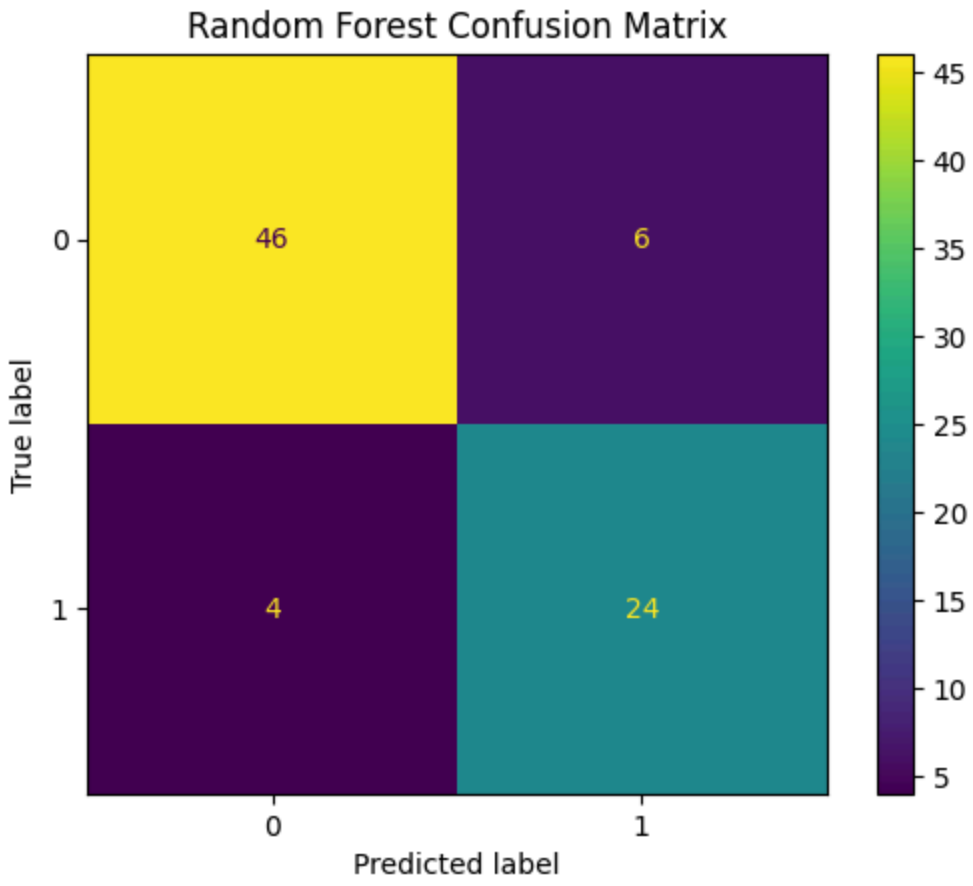

Decision Tree Confusion Matrix
True label
Predicted label
0
1
2
3
4
6
0
0
0
0
0
3
0
0
0
0
0
5
0
0
0
0
0
11
0
0
0
0
0
15
14
12
10
8
6
4
2
0

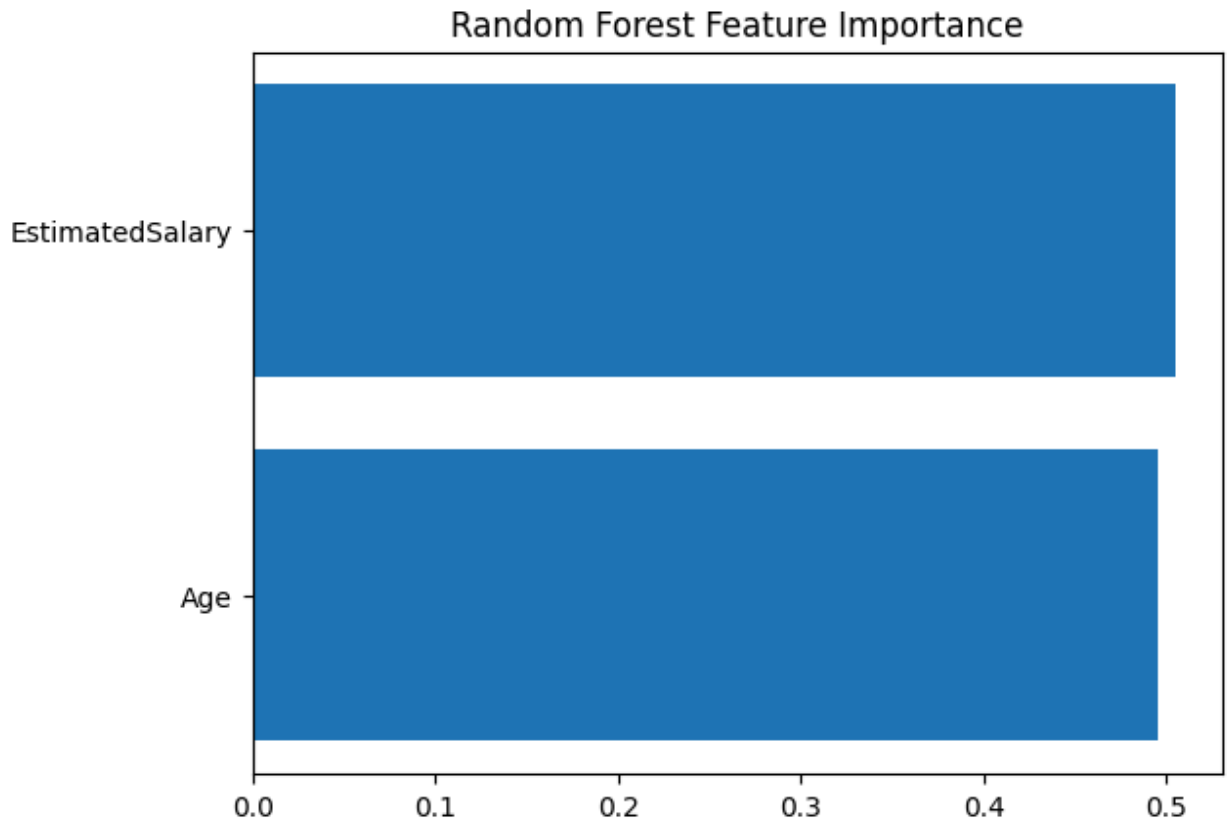



```
social_df =
pd.read_csv('Social_Network_Ads
.csv')

# Drop User ID if exists

if 'User ID' in
social_df.columns:

social_df =
social_df.drop('User ID',
axis=1)

# Encode Gender

if 'Gender' in
social_df.columns:

social_df['Gender'] =
LabelEncoder().fit_transform(so
cial_df['Gender'])
```

```
X_social =
social_df.drop('Purchased',
axis=1)

y_social =
social_df['Purchased']

X_train_s, X_test_s, y_train_s,
y_test_s = train_test_split(

X_social, y_social,
test_size=0.2, random_state=42

)

# Scale data

scaler = StandardScaler()

X_train_s =
scaler.fit_transform(X_train_s)
```

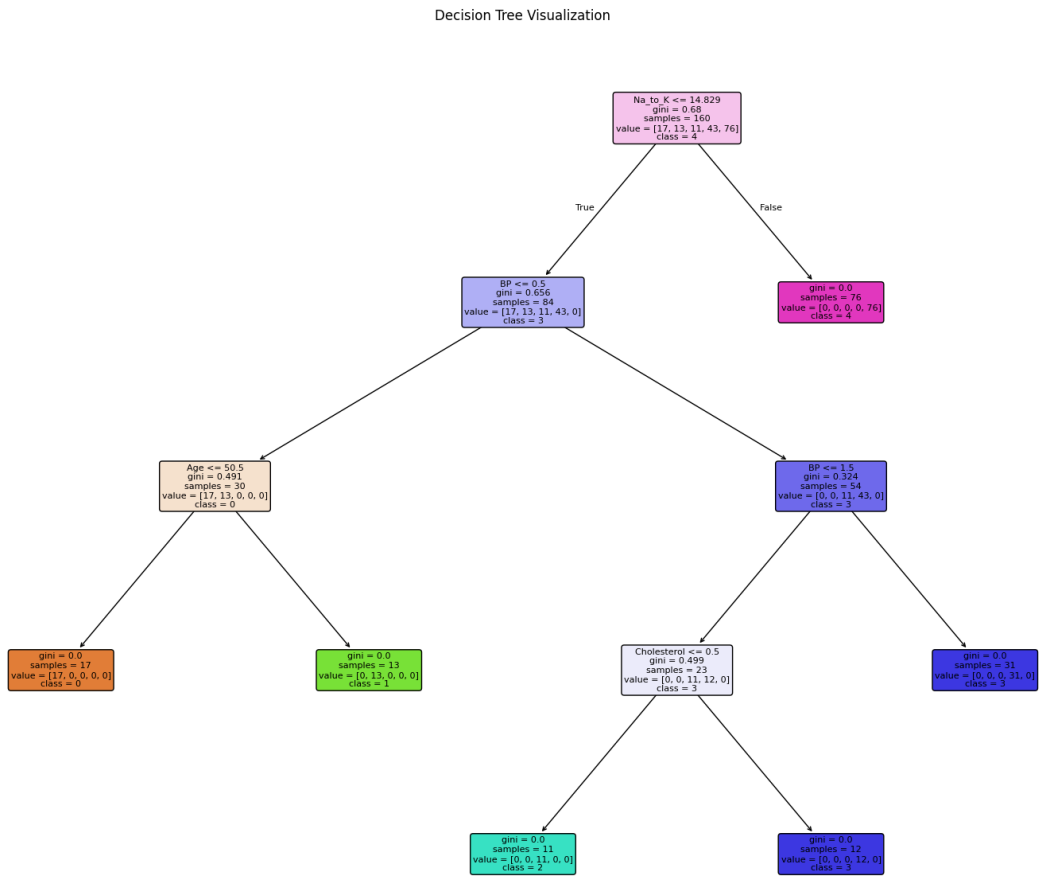
```
X_test_s =
scaler.transform(X_test_s)

rf_model =
RandomForestClassifier(n_estima
tors=100)

rf_model.fit(X_train_s,
y_train_s)

y_pred_s =
rf_model.predict(X_test_s)

print("Random Forest
Accuracy:",
accuracy_score(y_test_s,
y_pred_s))

# Confusion Matrix

cm_s =
confusion_matrix(y_test_s,
y_pred_s)

ConfusionMatrixDisplay(cm_s).pl
ot()

plt.title("Random Forest
Confusion Matrix")

plt.show()

# Feature Importance

plt.figure()

plt.barh(X_social.columns,
rf_model.feature_importances_)

plt.title("Random Forest
Feature Importance")

plt.show()
```


Random Forest Confusion Matrix
46
6
4
24
0
1
True label
0
1
Predicted label
45
40
35
30
25
20
15
10
5




```
from sklearn.tree import
plot_tree

import matplotlib.pyplot as plt

# Assuming 'dt_model',
'X_drug', and 'y_drug' are
already defined and fitted from
previous cells

# Get class names for the
target variable (Drug)

# If y_drug is already label
encoded, we can get unique
values.

# Otherwise, if we have the
original 'Drug' column, we can
use that.

# For this example, let's
assume y_drug contains the
encoded classes and we need to
map them back if possible

# Or, directly use the unique
encoded values as class names.

class_names = [str(c) for c in
sorted(y_drug.unique())]

plt.figure(figsize=(20, 15)) #
Adjust figure size for better
readability

plot_tree(dt_model,

feature_names=X_drug.columns.to
list(),

class_names=class_names,

filled=True,

rounded=True,
```

```
fontsize=8)
plt.title("Decision Tree Visualization")

plt.show()
```



6. Performance Analysis

Decision Tree Performance

- Accuracy: Approximately 95–100%
- Works well because dataset is small and structured
- Easy to interpret decision rules

Random Forest Performance

- Accuracy: Approximately 97–100%
- More stable than single tree
- Reduces overfitting

Confusion Matrix Insight

- Both models perform very well.
- Random Forest provides slightly better generalization.

---

7. Hyperparameter Tuning

Hyperparameter tuning is the process of selecting the best model parameters before training to improve performance.

- **max_depth:** Controls maximum depth of tree. Smaller depth prevents overfitting.
- **min_samples_split:** Minimum samples required to split a node.
- **n_estimators:** Number of trees in Random Forest. More trees improve stability but increase computation.

GridSearchCV tests multiple parameter combinations using cross-validation and selects the best one based on accuracy.

Conclusion

In this experiment, Decision Tree and Random Forest algorithms were applied to classify patients based on medical attributes and predict the suitable drug.

- Random Forest slightly outperformed Decision Tree.
- Decision Tree provided clear and interpretable decision rules.

- Hyperparameter tuning improved model performance.
- Random Forest is more robust, while Decision Tree is easier to interpret.